

Machine Learning I
80-629A

Apprentissage Automatique I
80-629

Recommender Systems
— Week #11

Introduction

- Recommendation task:
 - Suggest items of interest to users
 - Items: movies, books, articles, humans
 - Users: humans

Is it worth our attention?

- Recommendation is the next search
 - Search finds items (given a query)
 - Recommendation finds items of interest

Is it worth our attention?

new

- Recommendation is the ~~next~~ search
- Search finds items (given a query)
- Recommendation finds items of interest

Frequently Bought Together



+



+



Price for all three: \$230.47

 Add all three to Cart

Add all three to Wish List

Show availability and shipping details

- ☒ **This item:** Pattern Recognition and Machine Learning (Information Science and Statistics) by Christopher M. Bishop
Hardcover **\$64.01**
- ☒ **Machine Learning: A Probabilistic Perspective (Adaptive Computation and Machine Learning series)** by Kevin P. Murphy
Hardcover **\$78.26**
- ☒ **Probabilistic Graphical Models: Principles and Techniques (Adaptive Computation and Machine Learning ...** by Daphne Koller
Hardcover **\$88.20**

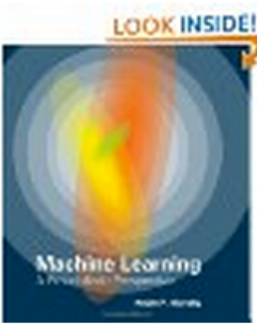
Customers Who Bought This Item Also Bought

Page 1





Learning From Data
› Yaser S. Abu-Mostafa
★★★★★ (56)
Hardcover



Machine Learning: A Probabilistic ...
› Kevin P. Murphy
★★★★★ (29)
Hardcover



The Elements of Statistical Learning: ...
Trevor Hastie
★★★★★ (31)
Hardcover



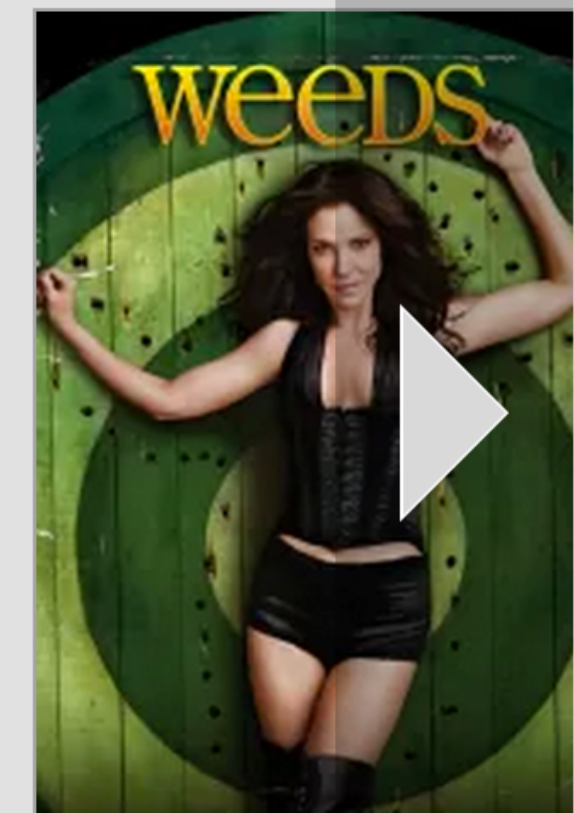
Probabilistic Graphical Models: Principles ...
› Daphne Koller
★★★★★ (24)
Hardcover



Machine Learning
› Tom M. Mitchell
★★★★★ (47)
Hardcover
\$195.71 



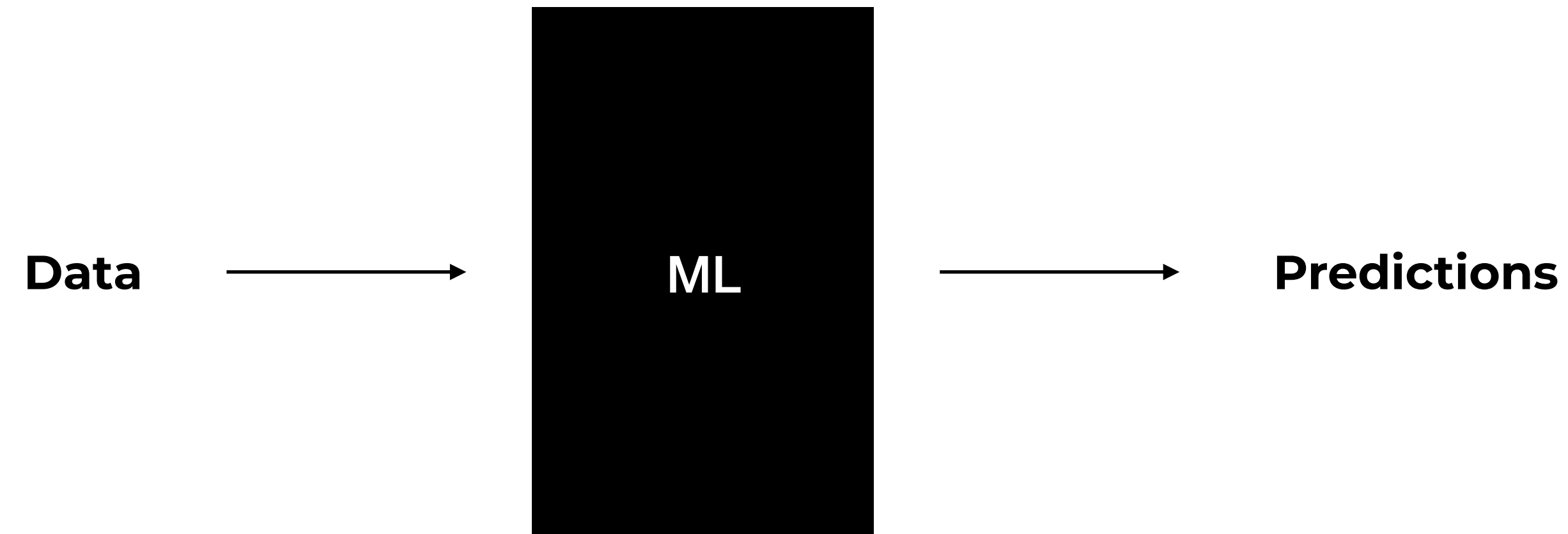
Top Picks for Me



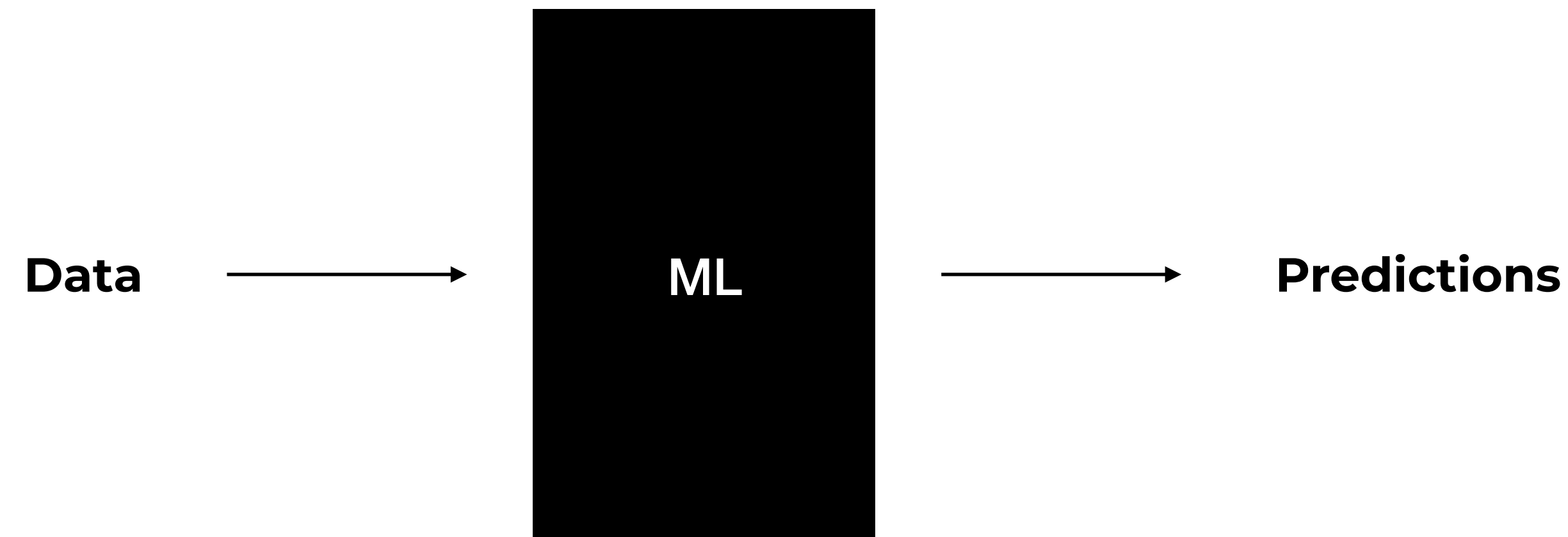
Machine Learning for Recommender Systems

- Task: Suggest items of interest to users
- From data how do you determine what denotes interest?
 - Item-specific signal (supervised learning)
 1. Score: rating, bid
 2. Consumption: click, buy, watch, bookmark

- **Imagine**
 - The data are user ratings
 - Task: Recommend items the user will like
- How do we set it up as a machine learning problem?



- Imagine
 - The data are user ratings
 - Task: Recommend items the user will like
- How do we set it up as a machine learning problem?



- **Task:** What do we learn? What do we predict? What is the model?
- **Performance measure:** How do we evaluate the results?
- **Experience:** How does our model interact with data?

Framework for recommendation problems

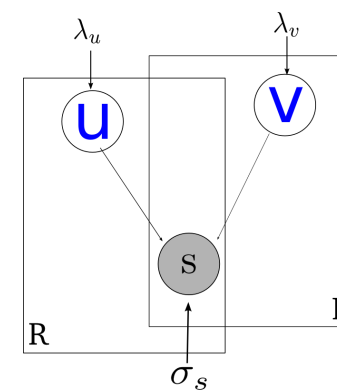


- User preferences

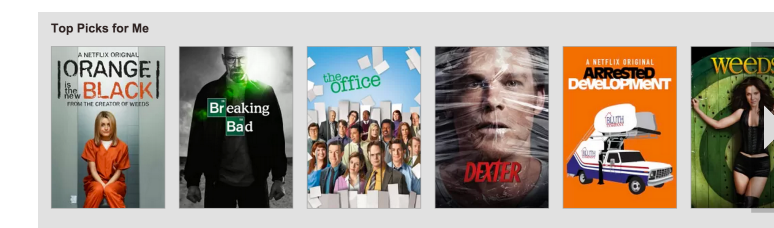


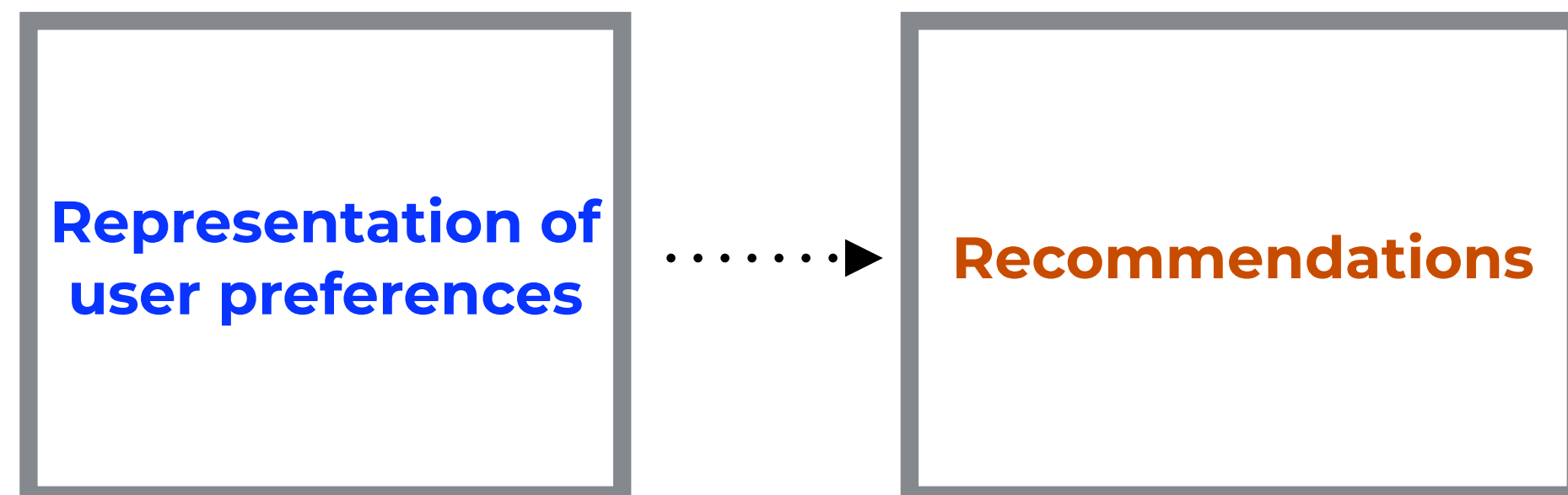
- User/Item features

- Model



- E.g. Top-N recommendations





- Task:
 - How we we set it up?
 1. Regression (Classification)
 2. Ranking

Ranking vs. Regression

A. Ranking models

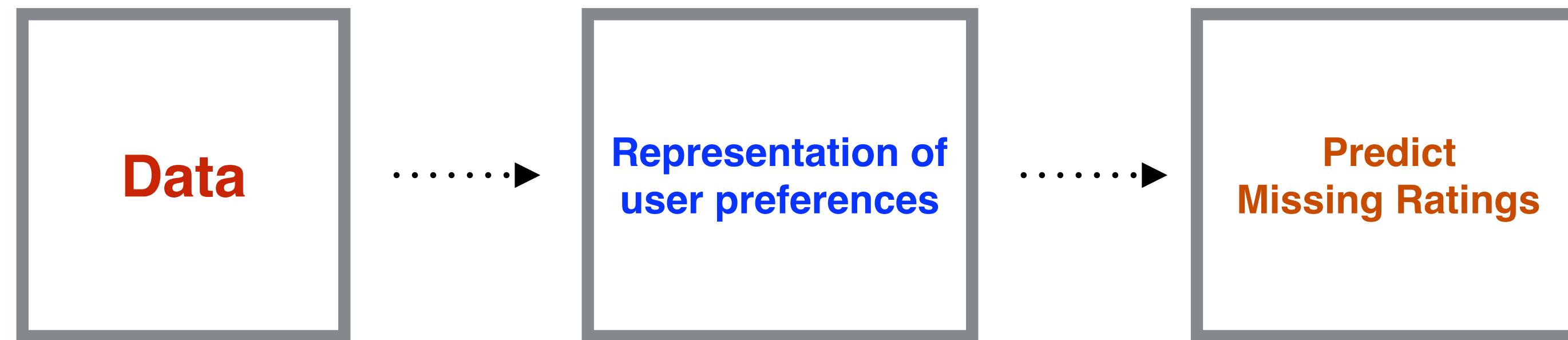
- Computationally more expensive
- E.g., Have to consider a group of items (listwise)

$$f : \underbrace{(u, i_1, i_2, \dots, i_m)}_{\text{user } u\text{'s unseen items}} \rightarrow \underbrace{(r_1, r_2, \dots, r_m)}_{\text{rank of each item}}$$

B. Score models

- For each user:
 1. Predict scores of all unseen items $f : (u, i) \rightarrow \mathbb{R}$
 2. Rank items (show top-K)

Framework for recommendation problems

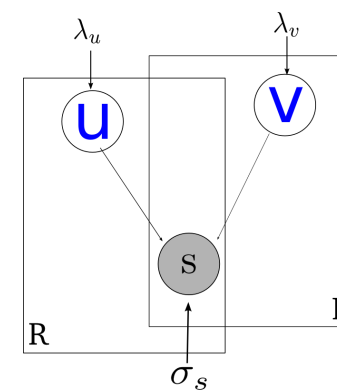


- **User preferences**



- **User/Item features**

- **Model**



Score Prediction as regression

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}_{\text{users} \times \text{items}} \xrightarrow{f} \begin{bmatrix} 3 & 2 & \dots & 0 \\ 1 & 0 & \dots & 3 \\ \vdots & & \ddots & \dots \\ 0 & 2 & \dots & 2 \end{bmatrix}$$

Train: Black S^o
Test : Red S^u

Modelling?

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}_{\text{users} \times \text{items}} \xrightarrow{f} \begin{bmatrix} 3 & \color{red}{2} & \dots & 0 \\ \color{red}{1} & 0 & \dots & \color{red}{3} \\ \vdots & & \ddots & \dots \\ 0 & \color{red}{2} & \dots & \color{red}{2} \end{bmatrix}$$

- How do we set this up as a learning problem?

$$S^u = f(S^o)$$

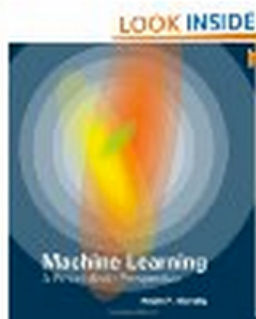
Collaborative Filtering (CF)

- Assumption:
 - Users with past similar preferences will have similar future preferences
- Work horse used in many recommender systems

Customers Who Bought This Item Also Bought

Page 1



				
Learning From Data › Yaser S. Abu-Mostafa	Machine Learning: A Probabilistic ... › Kevin P. Murphy	The Elements of Statistical Learning: ... Trevor Hastie	Probabilistic Graphical Models: Principles ... › Daphne Koller	Machine Learning › Tom M. Mitchell
★★★★★ (56)	★★★★★ (29)	★★★★★ (31)	★★★★★ (24)	★★★★★ (47)
Hardcover	Hardcover	Hardcover	Hardcover	Hardcover
	\$70.00	\$60.40	\$60.00	\$195.71 Prime

CF- Neighbourhood approaches

1. For each user, find other users with similar past preferences
2. Predict that user's missing preferences as the weighted combination of its neighbours' preferences

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \vdots \\ 2 & - & \dots & - \end{bmatrix}_{\text{users} \times \text{items}}$$

CF- Neighbourhood

1. Find similarity between every pair of users (or items)

$$\text{Sim}(u, u') = \frac{(s_u^o)^\top s_{u'}^o}{\|s_u^o\| \|s_{u'}^o\|}$$

$$\begin{matrix} s_{u'}^o \\ s_u^o \end{matrix} \begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \vdots \\ 2 & - & \dots & - \end{bmatrix}$$

2. Predict missing scores using a user's neighbours

$$\hat{S}_{uj} = \sum_{u'} \text{Sim}(u, u') s_{u'j}^o, \\ \forall u' \text{ that have rated } j$$

CF- Neighbourhood approaches

- Non-parametric approach
 - A user is represented by a weighted combination of its neighbours
 - New users can change one's recommendations
- Different distance functions to capture different effects
 - Ratings vs. clicks
 - Could consider additional information
- Works well empirically
- Building similarity matrix can be slow (offline)
- Not probabilistic

CF - Matrix factorization

$$\begin{array}{c} \text{users} \end{array} \begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix} \approx \begin{array}{c} \text{users} \end{array} \begin{bmatrix} k \\ \theta \end{bmatrix} \begin{array}{c} \text{items} \\ k \end{array} \begin{bmatrix} \beta \end{bmatrix}$$

Model. $S_{ui} := \theta_u^\top \beta_i$

Parameters. $\theta_u \forall u, \beta_i \forall i$

Objective. $\sum_u \sum_i (S_{ui} - \hat{S}_{ui})^2$

- Assumption: the observation matrix is low-rank
- Estimates user and item representations
- k is a hyperparameter

$$k \ll \min(|\text{Users}|, |\text{Items}|)$$

CF - Matrix factorization.

Alternative view.

Model. $S_{ui} := \theta_u^\top \beta_i$

Imagine that θ_u 's are features of users

The model is then a linear regression for each item:

$$\begin{aligned} S_{ui} &= \theta_u^\top \beta_i \\ &= \sum_k \theta_{uk} \beta_{ik} \\ &= \sum_k \theta_{u1} \beta_{i1} + \theta_{u2} \beta_{i2} + \dots + \theta_{up} \beta_{ip} \end{aligned}$$

Since the model is symmetric in θ and β ,
 β_i 's can be seen as features of items

Model fitting

Objective. $\sum_u \sum_i (\mathbf{s}_{ui} - \hat{\mathbf{s}}_{ui})^2$

Model fitting

Objective. $\sum_u \sum_i (\mathbf{s}_{ui} - \hat{\mathbf{s}}_{ui})^2$

- Joint parameter optimization
- Gradient descent: $(\nabla \theta, \nabla \beta)$

Model fitting

Objective. $\sum_u \sum_i (\mathbf{s}_{ui} - \hat{\mathbf{s}}_{ui})^2$

- Joint parameter optimization
 - Gradient descent: $(\nabla \theta, \nabla \beta)$
- Alternate optimization
 1. Fix θ , update β
 2. Fix β , update θ
- Each step is a (regularized) least-squares problem
- This procedure is known as alternating least squares (ALS)

Matrix Factorization

S^o

↓

↓

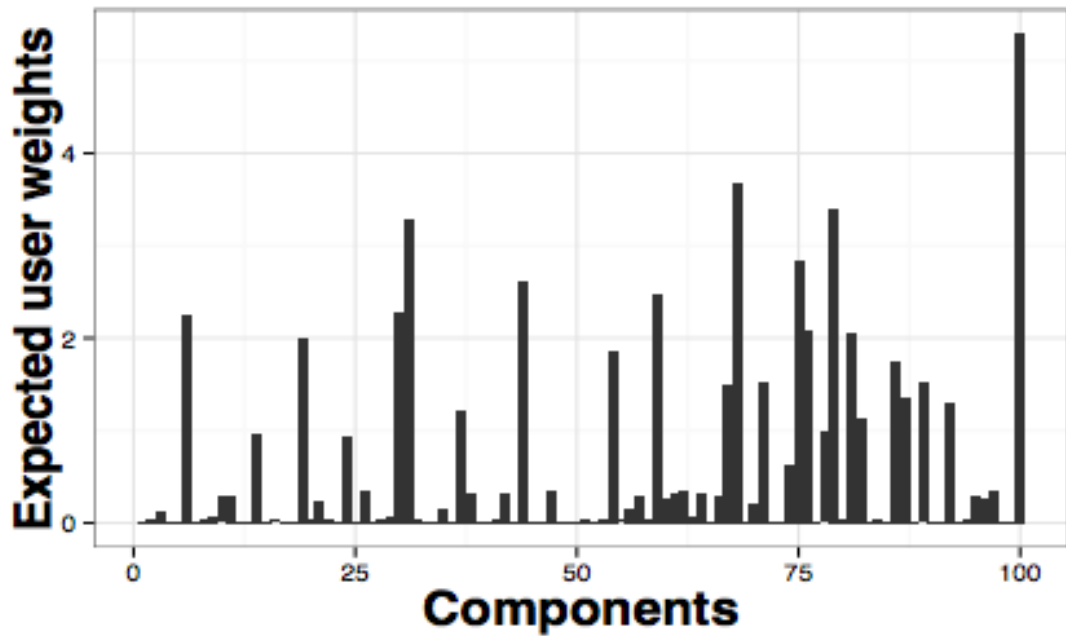
S^u

User's highly rated movies

- E.T. the Extra-Terrestrial (Children's, Drama)
- Full Metal Jacket (Action, Drama, War)
- Three Colors: Red (Drama)
- Breaker Morant (Drama, War)
- Shakespeare in Love (Comedy, Romance)
- Shadowlands (Drama, Romance)
- Rob Roy (Drama, Romance, War)
- The Verdict (Drama)
- A Little Princess (Children's, Drama)
- Leaving Las Vegas (Drama, Romance)



User's weights for 100 components



Top movies recommended for the user

- Casablanca (Drama, Romance, War)
- Breakfast at Tiffany's (Drama, Romance)
- Amadeus (Drama)
- When Harry Met Sally... (Comedy, Romance)
- American Beauty (Comedy, Drama)
- Fargo (Crime, Drama, Thriller)
- The Right Stuff (Drama)
- Gandhi (Drama)
- Apocalypse Now (Drama, War)
- Toy Story (Children's, Comedy, Animation)

Model Exporation

$\text{argsort}_i \beta_{ik}$

Movielen	"Sci-Fi"	"Drama, Romance"	"Action"	6K Users 4K Movies 1M Ratings
	Day the Earth Stood Still Metropolis Forbidden Planet Them! Invasion of the Body Snatchers The War of the Worlds Godzilla Village of the Damned Night of the Living Dead The Thing From Another World	Strictly Ballroom Like Water for Chocolate The Postman Sense and Sensibility Much Ado About Nothing The Remains of the Day Howards End An Ideal Husband Henry V Shawdowlands	Die Hard 2 Die Hard: With a Vengeance Independence Day Air Force One The Rock Con Air Enemy of the State Conspiracy Theory The Matrix Broken Arrow	
Netflix	"Supernatural thriller"	"Literary films"	"Friends sitcom"	480K Users 17.7K Movies 100M Ratings
	Stir of Echoes The Exorcist The Ring Final Destination Misery What Lies Beneath Poltergeist The Shining Carrie Gothika	Pride and Prejudice Sense and Sensibility Elizabeth Emma Sense and Sensibility Mansfield Park Much Ado About Nothing The Importance of Being Earnest Anne of Green Gables Shakespeare in Love	Friends: Season 1 Friends: Season 2 Friends: Season 4 The Best of Friends: Vol. 1 Friends: Season 3 Friends: Season 5 The Best of Friends: Season 1 The Best of Friends: Season 2 The Best of Friends: Season 3 Friends: Season 6	
Mendeley	"Sociology"	"Wireless sensor networks"	"Distributed behavior"	80K Users 260K Sci. articles 100M Ratings
	Social Capital: Its Origins, Institutions and Economic... Institutions and Economic... Increasing Returns and Path Dependence... Diplomacy & Domestic Polictics... Comparative Polictics and the Comparative.. Ethnicity, Insurgency, and Civil War... Historical Institutionalism in Comparative... Case studies and theory development in social... The Politics, Power, Pathologies.. End of the Transition Paradigm...	Wireless sensor networks: a survey... Wireless sensor network survey An energy-efficient MAC protocol.. A survey of routing protocols for.. Wireless sensor networks for habitat.. Cognitive radio: brain-empowered wireless.. A survey on wireless multimedia sensor networks NeXt generation/dynamic spectrum... Routing techniques in wireless sensor... Social network analysis...	Flocks, herds and schools Flocking for multi-agent.. Market-Based multirobot... Coordination of groups of mobile autonomous... Behavior-based formation control for multi robot teams... A formal analysis and taxonomy of task allocation... A survey of consensus problem in multi-agent coordination... Modeling swarm robotic systems:... Cooperative mobile robotics: A case study... The e-puck, a robot designed for education in engineering...	

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix} \longrightarrow \begin{array}{l} (\text{User 1, Item 1, 3}) \\ (\text{User 1, Item 2, 0}) \\ (\text{User 2 Item 2, 0}) \\ (\text{User n, Item 1, 2}) \end{array}$$

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$



(User 1, Item 1, 3)
 (User 1, Item 2, 0)
 (User 2, Item 2, 0)
 (User n, Item 1, 2)

Encode each categorical variable
 using a series of indicator variables



User Item

(1 0 ... 0, 1 0 ... 0, 3)
 (1 0 ... 0, 0 0 ... 1, 0)
 (0 1 ... 0, 0 1 ... 1, 0)
 (0 0 ... 1, 1 0 ... 0, 2)

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$

(User 1, Item 1, 3)
 (User 1, Item 2, 0)
 (User 2, Item 2, 0)
 (User n, Item 1, 2)

Encode each categorical variable
 using a series of indicator variables

User Item
 (1 0 ... 0, 1 0 ... 0, 3)
 (1 0 ... 0, 0 0 ... 1, 0)
 (0 1 ... 0, 0 1 ... 1, 0)
 (0 0 ... 1, 1 0 ... 0, 2)

$$\underbrace{(1 \ 0 \ \dots \ 0, 1 \ 0 \ \dots \ 0)}_x, \underbrace{3}_y$$

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix} \longrightarrow$$

(User 1, Item 1, 3)
 (User 1, Item 2, 0)
 (User 2, Item 2, 0)
 (User n, Item 1, 2)

Encode each categorical variable
 using a series of indicator variables

User Item

(1 0 ... 0, 1 0 ... 0, 3)
 (1 0 ... 0, 0 0 ... 1, 0)
 (0 1 ... 0, 0 1 ... 1, 0)
 (0 0 ... 1, 1 0 ... 0, 2)

$$\underbrace{(1 \text{ } 0 \text{ } \dots \text{ } 0, 1 \text{ } 0 \text{ } \dots \text{ } 0, 3)}_x$$

$$\underbrace{\quad}_{x1} \quad \underbrace{\quad}_{x2} \quad \underbrace{\quad}_y$$

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$



(User 1, Item 1, 3)
 (User 1, Item 2, 0)
 (User 2, Item 2, 0)
 (User n, Item 1, 2)

Encode each categorical variable
 using a series of indicator variables



User Item

(1 0 ... 0, 1 0 ... 0, 3)
 (1 0 ... 0, 0 0 ... 1, 0)
 (0 1 ... 0, 0 1 ... 1, 0)
 (0 0 ... 1, 1 0 ... 0, 2)

$$\underbrace{(1 \ 0 \ \dots \ 0, 1 \ 0 \ \dots \ 0, 3)}_x$$

$$\underbrace{\quad}_{x1} \quad \underbrace{\quad}_{x2}$$



$$y = (\mathbf{x}_1^\top \theta)^\top (\mathbf{x}_2^\top \beta)$$

Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$

(User 1, Item 1, 3)
(User 1, Item 2, 0)
(User 2, Item 2, 0)
(User n, Item 1, 2)

Encode each categorical variable
using a series of indicator variables

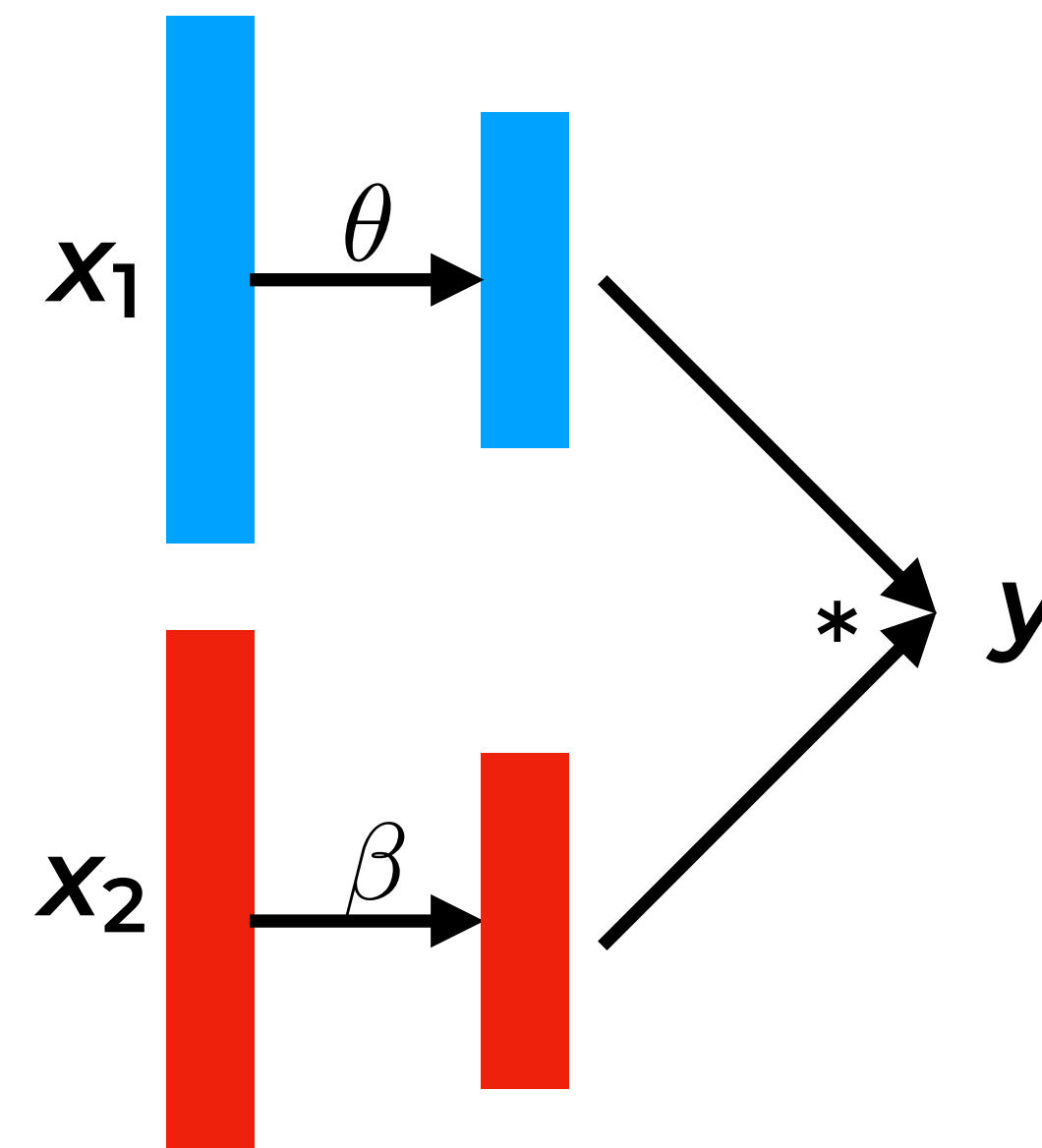
User Item

(1 0 ... 0, 1 0 ... 0, 3)
(1 0 ... 0, 0 0 ... 1, 0)
(0 1 ... 0, 0 1 ... 1, 0)
(0 0 ... 1, 1 0 ... 0, 2)

$$\underbrace{(1 \ 0 \ \dots \ 0, 1 \ 0 \ \dots \ 0)}_x, \underbrace{3}_y$$

$x_1 \quad x_2$

$$y = (\mathbf{x}_1^\top \theta)^\top (\mathbf{x}_2^\top \beta)$$



Towards CF with deep learning

$$\begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix}$$

(User 1, Item 1, 3)
(User 1, Item 2, 0)
(User 2, Item 2, 0)
(User n, Item 1, 2)

Encode each categorical variable
using a series of indicator variables

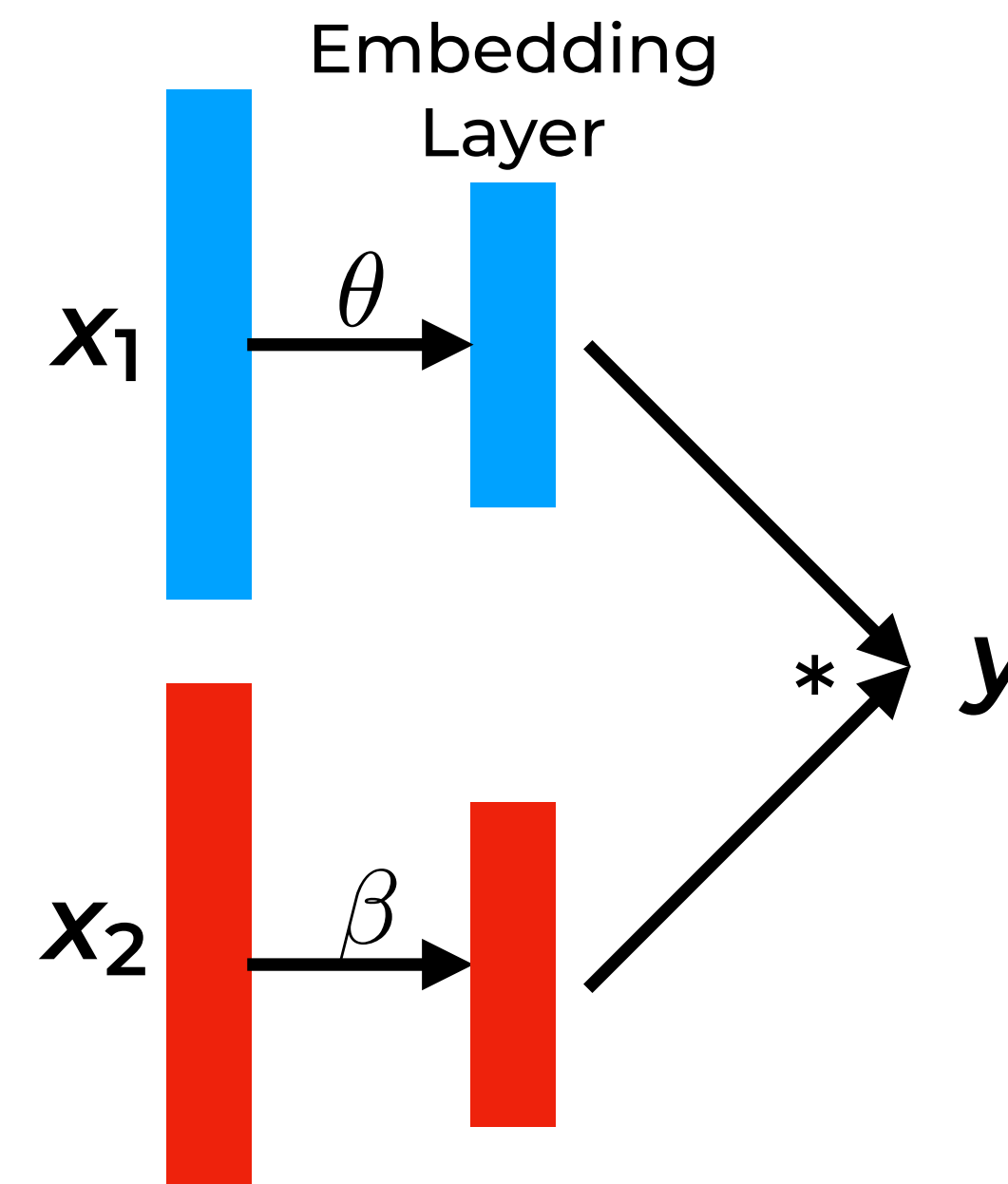
User Item

(1 0 ... 0, 1 0 ... 0, 3)
(1 0 ... 0, 0 0 ... 1, 0)
(0 1 ... 0, 0 1 ... 1, 0)
(0 0 ... 1, 1 0 ... 0, 2)

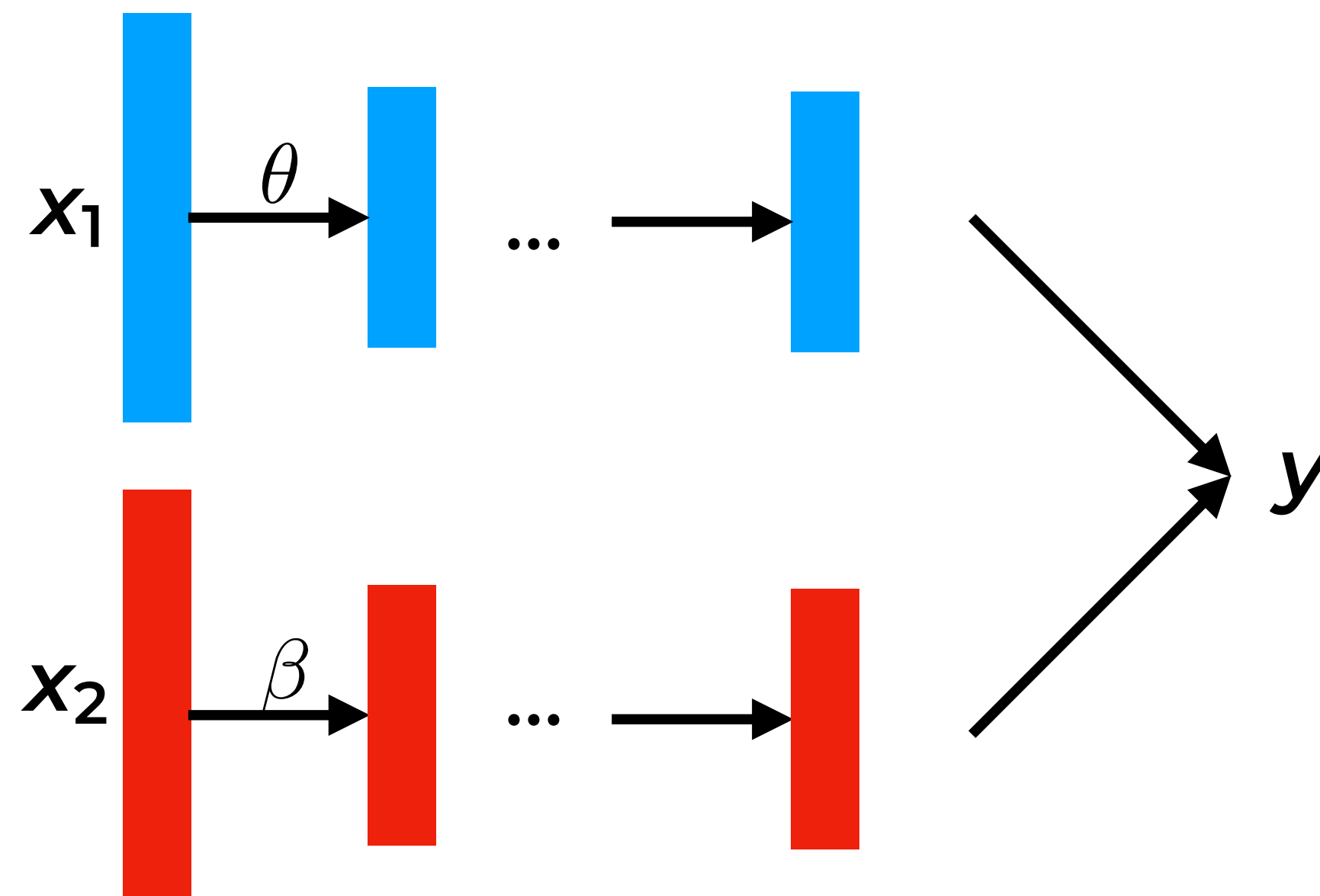
$$\underbrace{(1 \ 0 \ \dots \ 0, 1 \ 0 \ \dots \ 0)}_x, \underbrace{3}_y$$

$x_1 \quad x_2$

$$y = (\mathbf{x}_1^\top \theta)^\top (\mathbf{x}_2^\top \beta)$$



A version of Deep Matrix Factorization



- Can do more complicated user and item combinations (beyond dot product)

Autoencoders

- Popular neural-network model often used in unsupervised learning (e.g., dim reduction)

Autoencoders

- Popular neural-network model often used in unsupervised learning (e.g., dim reduction)
- Non-linear PCA

Autoencoders

- Popular neural-network model often used in unsupervised learning (e.g., dim reduction)
- Non-linear PCA
- Intuition: let's learn to copy the data

$$x' = f(x)$$

Autoencoders

- Popular neural-network model often used in unsupervised learning (e.g., dim reduction)
- Non-linear PCA
- Intuition: let's learn to copy the data

$$x' = f(x)$$

- We force a “bottleneck”

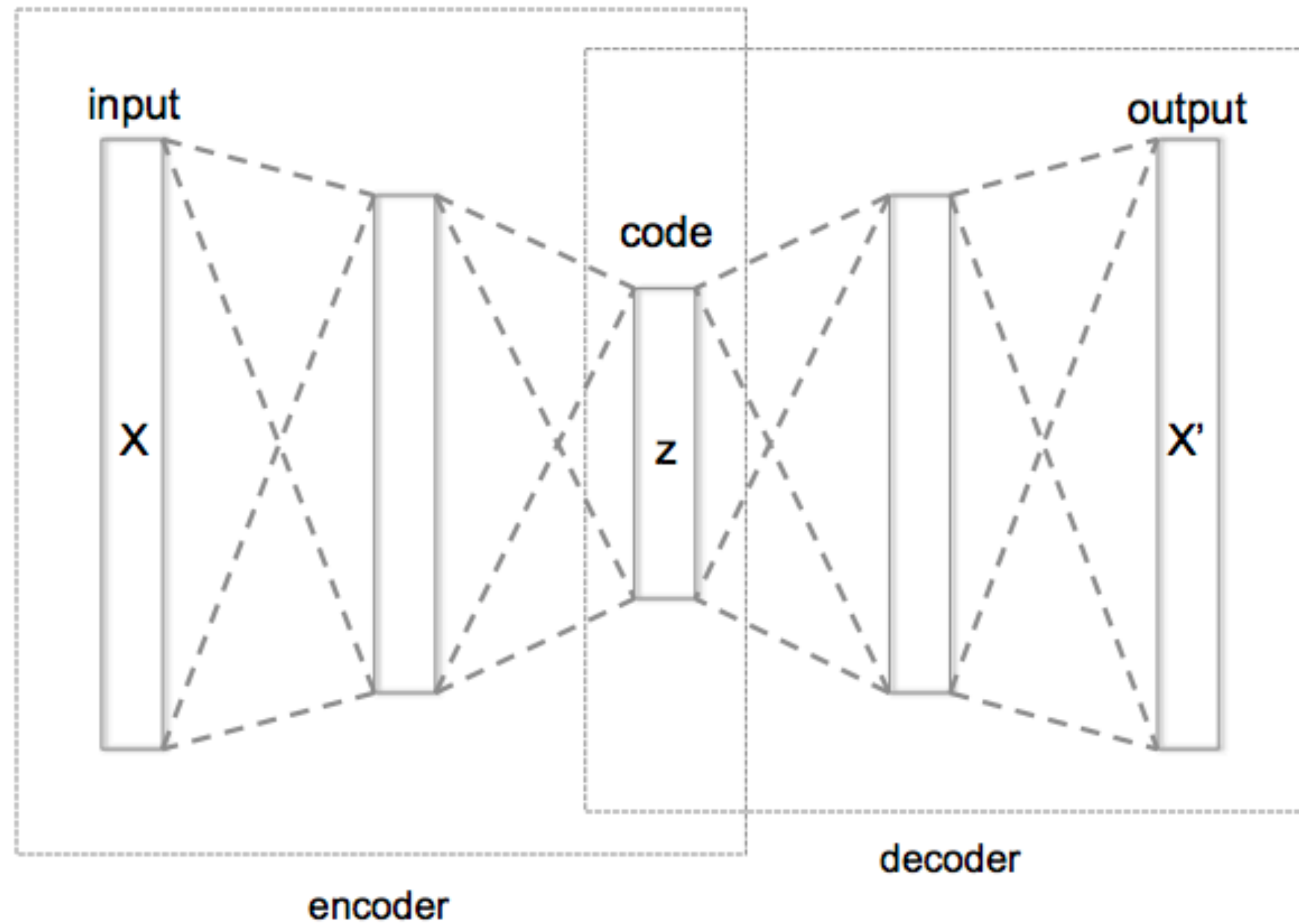
$$z = f_1(x)$$

$$x' = f_2(z)$$

$$\dim(z) < \dim(x)$$

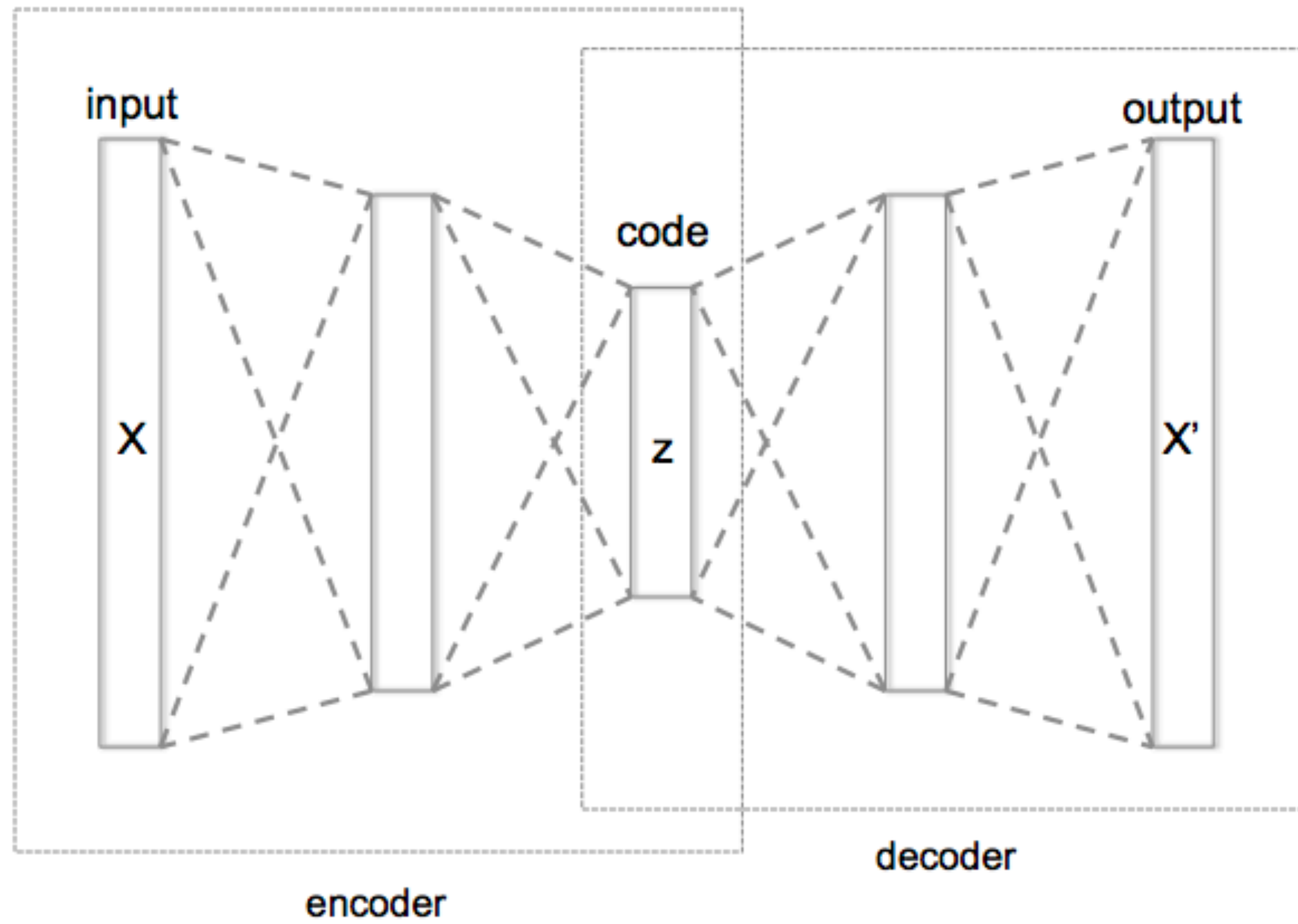
Autoencoders

$$\begin{aligned} \mathbf{z} &= \mathbf{f}_1(\mathbf{x}) \\ \mathbf{x}' &= \mathbf{f}_2(\mathbf{z}) \\ \dim(\mathbf{z}) &< \dim(\mathbf{x}) \end{aligned}$$



[From wikipedia]

Autoencoders for CF



[From wikipedia]

- X : Either a row or a column of the ratings matrix

$$X = \begin{bmatrix} 3 & - & \dots & 0 \\ - & 0 & \dots & - \\ \vdots & & \ddots & \dots \\ 2 & - & \dots & - \end{bmatrix} \text{ users} \times \text{items}$$

- Missing entries:
 - Set to 0 in the input
 - Not considered in the output
- Many versions using denoising-autoencoders (DAEs), VAEs, different likelihoods (etc.)

MF vs. AE for CF

- AE are more “naturally” non-linear
- AE are asymmetric
 - must choose whether to model users or items
- Versions of AEs are close to the state-of-the-art today

How to choose the right model?

- Search for papers that compare different models
- Keep a healthy dose of scepticism
 - I.e., results in papers is not necessarily ground truth
- Try it out on your data
 - Compare performance on held-out data
 - Can it handle: your data size, service speed, updating schedule, other desiderata (fairness, uncertainty estimates) ...

Are deep models better?

- 2014: No
- 2018: Yes
- 2019: Maybe not ... *
 - “Embarrassingly Shallow Autoencoders for Sparse Data”, Steck’19
 - “On the Difficulty of Evaluating Baselines: A Study on Recommender Systems”, Rendle et al.’19:
 - “With a careful setup of a **vanilla matrix factorization** baseline, we are not only able to improve upon the reported results for this baseline but even outperform the reported results of any newly proposed method.”

(* This is not considering possibly available covariates)

Explicit vs. Implicit data

Explicit vs. Implicit data

- Up to now we assumed ratings
- Ratings are **explicit data**:
 - “Users explicitly provide their preferences”
 - A high rating means the user liked the item
 - A low rating means the user disliked the item

Explicit vs. Implicit data

- Up to now we assumed ratings
- Ratings are **explicit data**:
 - “Users explicitly provide their preferences”
 - A high rating means the user liked the item
 - A low rating means the user disliked the item
- In practice implicit data is much more common:
 - click, buy, watch, listen

Challenge with Implicit Data

- Consuming an item usually implies a positive preference
- Not Consuming an item may either indicate:
 - A. A negative preference
 - B. Something else: e.g., lack of exposure or time

Challenge with Implicit Data

- The preference matrix is “full” (as opposed to sparse)
- ‘1’ indicates a consumed item
- ‘0’ indicates an unconsumed item

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

- You must take both 0s and 1s into account
- In practice many models can be adapted to implicit case

Common strategy for implicit data

- Model the 0s as being “less certain” than the 1s

$$\text{Objective MF: } \frac{1}{|\text{users}|} \sum_u \sum_i (S_{ui} - \hat{S}_{ui})^2$$

$$\text{Objective WMF: } \frac{1}{|\text{users}|} \sum_u \sum_i c_{ui}(S_{ui})(S_{ui} - \hat{S}_{ui})^2$$

$$c_{ui}(0) < c_{ui}(1)$$

- **Weighted Matrix Factorization** [Hu et al. '08]
- Learn the weight of each zero
- **Exposure Matrix Factorization** [Liang et al.'15]

User/Item features (I)

- Often additional information exists

User/Item features (I)

- Often additional information exists
 - Users: demographic information, social networks

User/Item features (I)

- Often additional information exists
 - Users: demographic information, social networks
 - Items: content (e.g., movie genre/trailer, book text)
 - Users & items:

User/Item features (I)

- Often additional information exists
 - Users: demographic information, social networks
 - Items: content (e.g., movie genre/trailer, book text)
- Users & items:
 - timestamps, session information

User/item features (II)

- **Allow for content-based recommendations**

User/item features (II)

- **Allow for content-based recommendations**
- **Good to combat the cold-start problem**

User/item features (II)

- **Allow for content-based recommendations**
 - **Good to combat the cold-start problem**
- **Assume that features are predictive of preferences**

User/item features (II)

- **Allow for content-based recommendations**
 - **Good to combat the cold-start problem**
- **Assume that features are predictive of preferences**
 - **More difficult in some domains than others (e.g., movies)**

User/item features (II)

- **Allow for content-based recommendations**
 - **Good to combat the cold-start problem**
- **Assume that features are predictive of preferences**
 - **More difficult in some domains than others (e.g., movies)**
- **A practical approach is to bootstrap with content-based to gather preference data and then switch to CF**

Evaluation

- Evaluate performance on held-out data (standard)
- Splitting data into train/validation/test:
 - Split by user to give equal “weight” to each user
 - Ensure that each user has enough data (no cold-start)

Evaluation metrics

1. Score prediction (explicit data only)

- Mean squared error: $\frac{1}{|\text{users}|} \sum_u \sum_i (s_{ui} - \hat{s}_{ui})^2$

Evaluation metrics

1. Score prediction (explicit data only)

- Mean squared error: $\frac{1}{|\text{users}|} \sum_u \sum_i (s_{ui} - \hat{s}_{ui})^2$

2. Information retrieval

- Precision, Recall
- Average rank, Mean average precision
- Normalized Discounted Cumulative Gain (NDCG)
 - Compares the ranking of your system with the optimal ranking
 - (Exponentially) Discounts items lower ranked items

Concluding Remarks (I)

- Type of models we have discussed are useful for:
 - Domains with large number of items (and users for CF)
 - Subjective preferences over attributes (features)
 - E.g., movies and not plane tickets
 - Items can be consumed relatively fast
 - E.g., restaurants/movies and not cars/houses

Concluding Remarks (II)

- CF models “work well” especially in large-data regimes
 - Commercial systems are reasonably good
 - There is evidence that companies derive value from them
- Much progress remains to be done
 - Modelling preferences is a very active research topic
 - Good preference models gave rise to other questions

Questions?

References

- Marital satisfaction and break-ups differ across on-line and off-line meeting venues, J. Cacioppo et al., PNAS'13
- A Probabilistic Model for Using Social Networks in Personalized Item Recommendation, A. Chaney et al., Recsys'15
- Deep Neural Networks for YouTube Recommendations, P. Covington et al., Recsys'16
- Content-based recommendations with Poisson factorization, P. Gopalan et al., NIPS'14
- Scalable Recommendation with Hierarchical Poisson Factorization., P. Gopalan et al., UAI'15
- Collaborative Filtering for Implicit Feedback Datasets, Hu et al., ICDM'08
- Exposure Matrix Factorization, D. Liang et al., WWW'16
- Factorization Machines, S. Rendle, ICDM '10
- Probabilistic Matrix Factorization, R. Salakhutdinov and A. Mnih, NIPS'08
- Session-based Social Recommendation via Dynamic Graph Attention Networks, W. Song et al., WSDM'19
- Collaborative Topic Modeling for Recommending Scientific Articles, C. Wang and D. Blei, KDD'11
- Deep matrix factorization models for recommender systems, HJ Xue et al., IJCAI'17